

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 5

Comprehensive Study of CNN Training, Regularization, Optimization, Hyperparameter Tuning, Transfer Learning and Cross-Validation

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to systematically study the effect of weight initialization, regularization, optimization algorithms, CNN hyperparameters, transfer learning, fine-tuning and cross-validation on image classification performance.

The experiment uses a single CNN architecture, MobileNetV2, and the Oxford-IIIT Pet dataset. Students will analyze the effect of individual design choices using training/validation curves and finally select a suitable configuration using 5-fold cross-validation.

2. Learning Outcomes

After completing this experiment, students will be able to:

- explain different weight initialization strategies;
- identify and analyze overfitting using training and validation curves;
- compare SGD, Momentum, RMSProp and Adam;
- explain CNN hyperparameters such as kernel size, stride, padding and pooling;
- understand Batch Normalization and Dropout;
- perform transfer learning and fine-tuning using MobileNetV2;
- tune important training hyperparameters;
- apply 5-fold cross-validation for model selection; and
- justify the final model using accuracy, variability and computational cost.

3. Dataset and Experimental Setup

Dataset: Oxford-IIIT Pet Dataset

The Oxford-IIIT Pet Dataset contains images of cats and dogs belonging to 37 breeds. The images are RGB and have different spatial dimensions. For this experiment, all images are resized to

$$224 \times 224 \times 3.$$

The dataset is suitable for demonstrating transfer learning using ImageNet-pretrained CNNs.

Experimental considerations:

- Use RGB images resized to 224×224 .
- Normalize the input images according to the pretrained model requirements.
- Maintain separate training, validation and test data.
- The test set must remain untouched during hyperparameter selection.
- Use MobileNetV2 because it is computationally lighter than many large CNN architectures and is more suitable for CPU-based laboratory work.



Figure 1: Sample Pet Images

The images show different breeds such as those from the Oxford-IIIT Pet dataset resized to 224x224. This confirms the dataset and labels are loaded correctly before training begins.

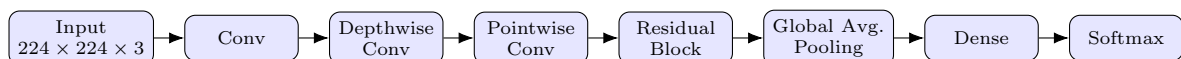
4. MobileNetV2 Architecture

MobileNetV2 is a lightweight CNN architecture designed to reduce computational complexity while maintaining good image representation.

Its important components include:

- depthwise convolution;
- pointwise 1×1 convolution;
- inverted residual blocks;
- linear bottlenecks;
- batch normalization; and
- ReLU6 activation.

A simplified representation is:



Note: The above diagram is a simplified conceptual representation rather than the complete MobileNetV2 architecture.

5. Weight Initialization

Weight initialization determines the initial values of trainable weights before training begins. A suitable initialization can improve convergence and reduce problems such as vanishing or exploding activations.

Students should investigate:

- Zero initialization
- Random initialization
- Xavier/Glorot initialization
- He initialization

Experimental Analysis

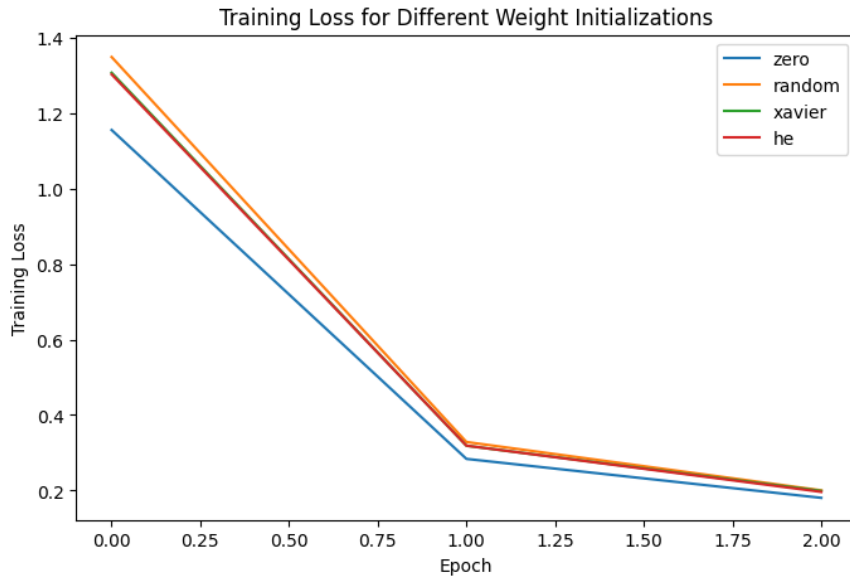


Figure 2: Training Loss for Different Weight Initializations

All four initialization methods show a decreasing training loss across epochs. Zero initialization and He initialization show the lowest final training loss among the methods tested.

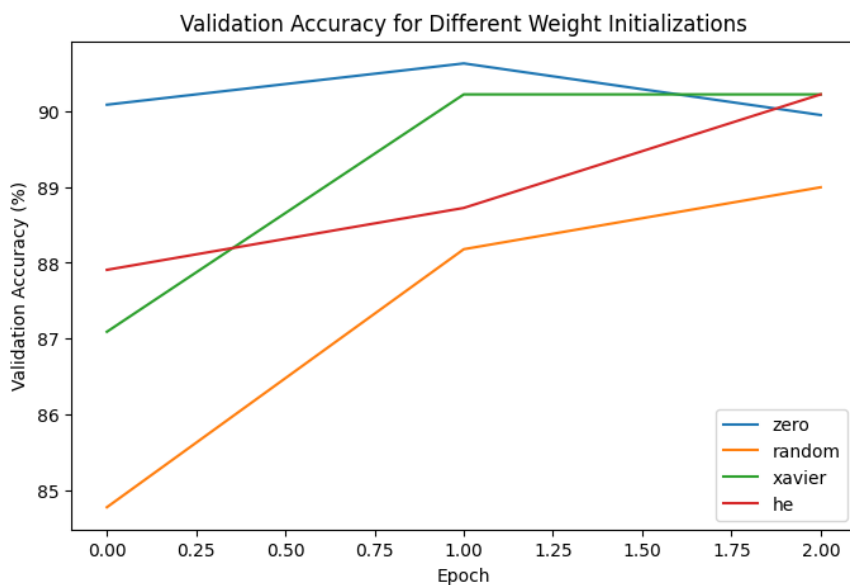


Figure 3: Validation Accuracy for Different Weight Initializations

Zero initialization achieved the highest validation accuracy at 90.63 percent, followed closely by Xavier and He initialization. Random initialization performed slightly lower at 88.99 percent, showing initialization method has a modest impact on this pretrained setup.

6. Regularization and Overfitting

Overfitting occurs when a model performs very well on the training data but generalizes poorly to unseen data. Students should compare:

- No regularization
- L2 regularization
- Dropout
- Batch Normalization

Experimental Analysis

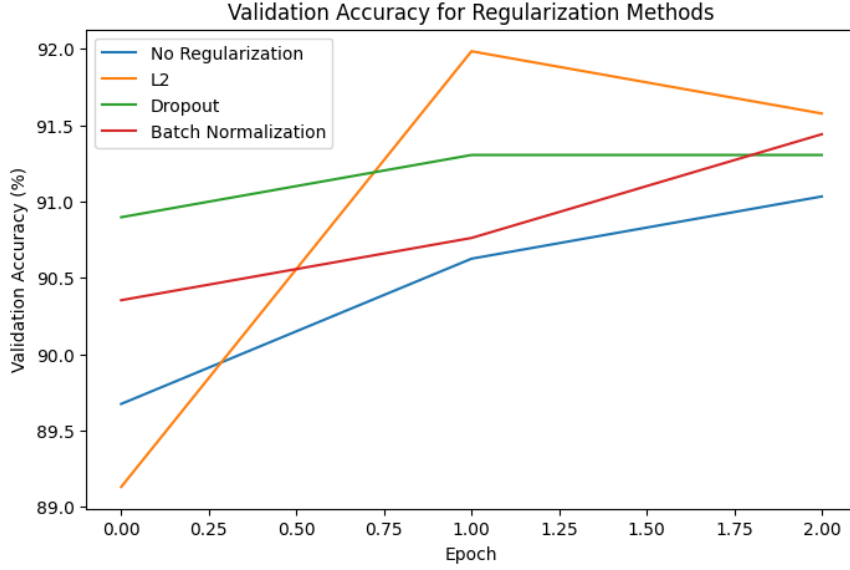


Figure 4: Validation Accuracy for Regularization Methods

L2 regularization achieved the highest validation accuracy at 91.98 percent among the four methods tested. Batch normalization also performed well, while dropout and no regularization showed slightly lower accuracy in comparison.

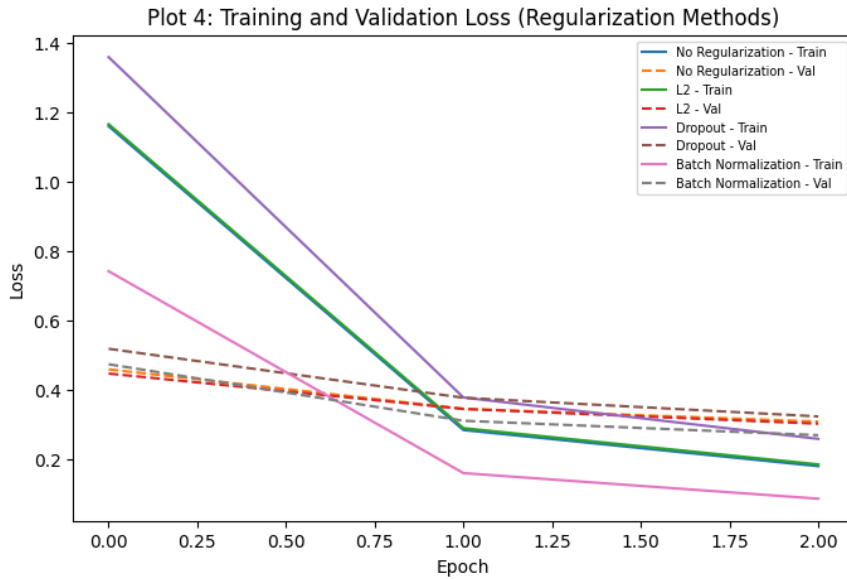


Figure 5: Training and Validation Loss for Regularization Methods

Batch normalization shows the lowest final training loss at 0.086, while dropout shows the highest at 0.258 due to its regularizing effect during training. Validation loss trends remain relatively close across all four methods.

7. Batch Normalization

Batch Normalization normalizes activations within a mini-batch during training. For a mini-batch

$$x_1, x_2, \dots, x_m,$$

the batch mean is

$$\mu_B = \frac{1}{m} \sum_{i=1}^m x_i$$

and the batch variance is

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (x_i - \mu_B)^2.$$

The normalized activation is

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}.$$

The final output is

$$y_i = \gamma \hat{x}_i + \beta,$$

where γ and β are learnable parameters.

Numerical Example

Consider

$$x = [2, 4, 6, 8].$$

The mean is

$$\mu_B = \frac{2 + 4 + 6 + 8}{4} = 5.$$

The variance is

$$\sigma_B^2 = \frac{(2 - 5)^2 + (4 - 5)^2 + (6 - 5)^2 + (8 - 5)^2}{4} = 5.$$

Therefore,

$$\sqrt{5} \approx 2.236.$$

Ignoring the very small ϵ ,

$$\hat{x}_1 = \frac{2 - 5}{2.236} \approx -1.342,$$

$$\hat{x}_2 = \frac{4 - 5}{2.236} \approx -0.447,$$

$$\hat{x}_3 = \frac{6 - 5}{2.236} \approx 0.447,$$

$$\hat{x}_4 = \frac{8 - 5}{2.236} \approx 1.342.$$

Hence,

$$\hat{x} \approx [-1.342, -0.447, 0.447, 1.342]$$

If $\gamma = 1$ and $\beta = 0$, these are the final outputs.

Inference: Batch Normalization produces approximately zero-centred and unit-scaled activations for this mini-batch. The learnable γ and β allow the network to learn an appropriate scale and shift.

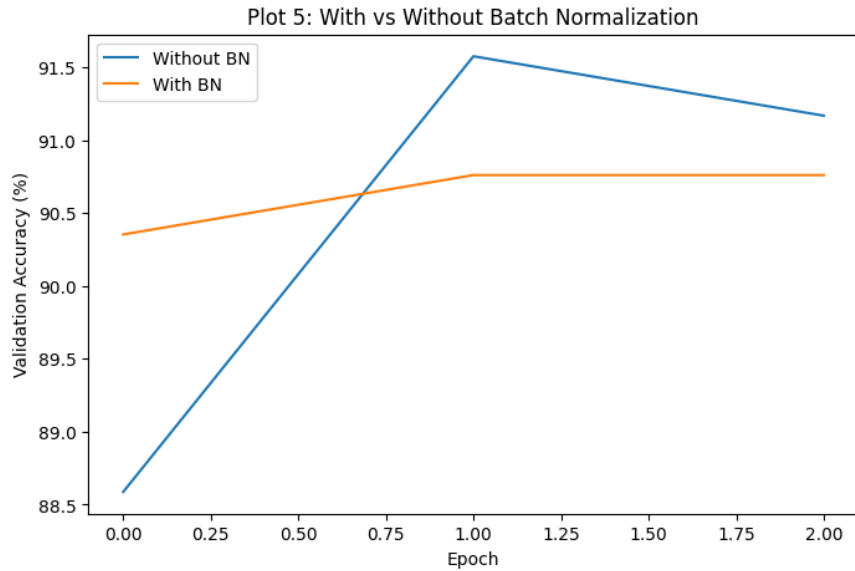


Figure 6: With vs Without Batch Normalization

The model without batch normalization achieved a slightly higher validation accuracy at 91.58 percent compared to 90.76 percent with batch normalization. This suggests batch normalization did not provide a clear benefit for this particular frozen base setup.

8. Optimization Algorithms

Students should compare the following optimization methods:

- SGD
- Momentum
- RMSProp
- Adam

The objective is to study convergence speed, stability and final validation performance.

Optimizer	Final Loss	Best Val. Accuracy	Epoch to Converge	Time
SGD	2.9545	83.83%	3	46.91 s
Momentum	0.7656	90.63%	3	47.64 s
RMSProp	0.1596	90.49%	3	46.57 s
Adam	0.1776	91.44%	3	47.39 s

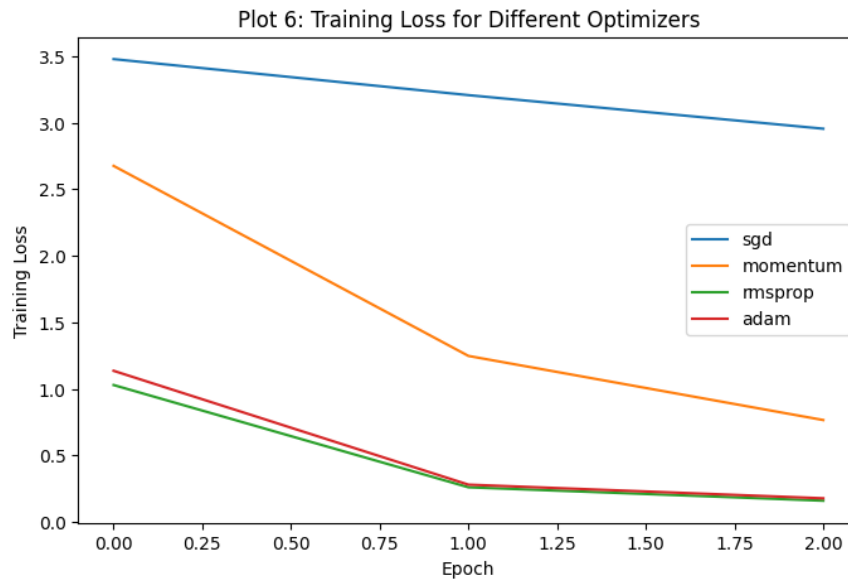


Figure 7: Training Loss for Different Optimizers

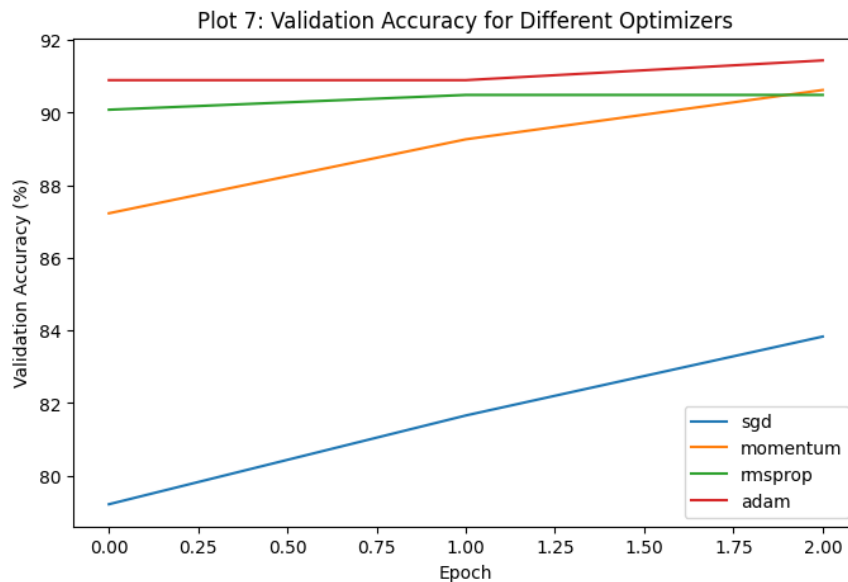


Figure 8: Validation Accuracy for Different Optimizers

Adam performed best at 91.44% with low loss 0.178. SGD lagged behind at 83.83% with high loss 2.95, showing it converges much slower without adaptive learning.

9. CNN Hyperparameter Tuning

The following hyperparameters should be investigated:

Hyperparameter	Suggested Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Dropout Rate	0, 0.25, 0.5
Optimizer	SGD, Adam
Fine-Tuning Learning Rate	10^{-4} , 10^{-5}
Frozen Layers	Frozen base / Partial unfreezing

For CNN concepts, students should also understand:

- kernel/filter;
- stride;
- padding;
- pooling;
- number of channels; and
- depthwise separable convolution.

For a convolution operation, the output dimension can be calculated as

$$O = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1,$$

where N is input size, K is kernel size, P is padding and S is stride.

Experimental rule: During a controlled hyperparameter study, change one hyperparameter at a time while keeping the remaining settings fixed.

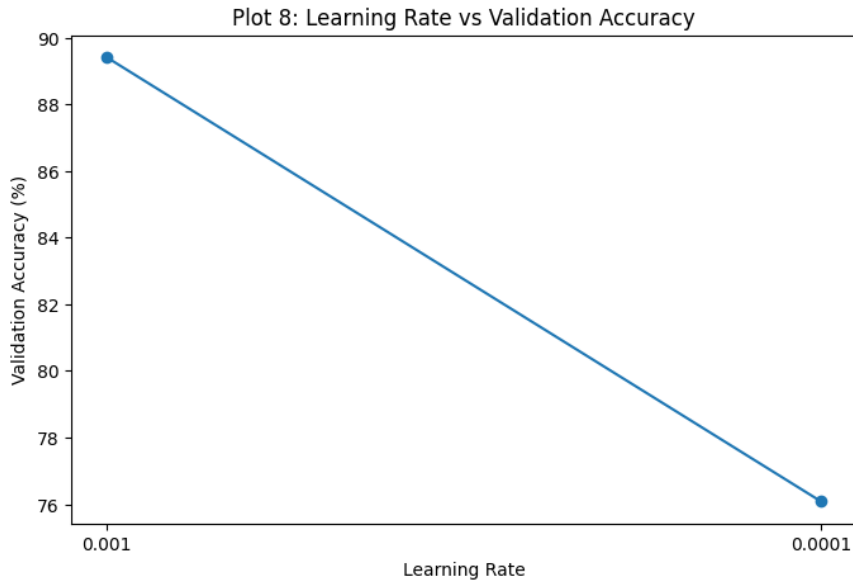


Figure 9: Learning Rate vs Validation Accuracy

Learning rate 0.001 gave 89.40% accuracy, while 0.0001 gave only 76.09%. The smaller rate was too slow to converge within the given epochs.

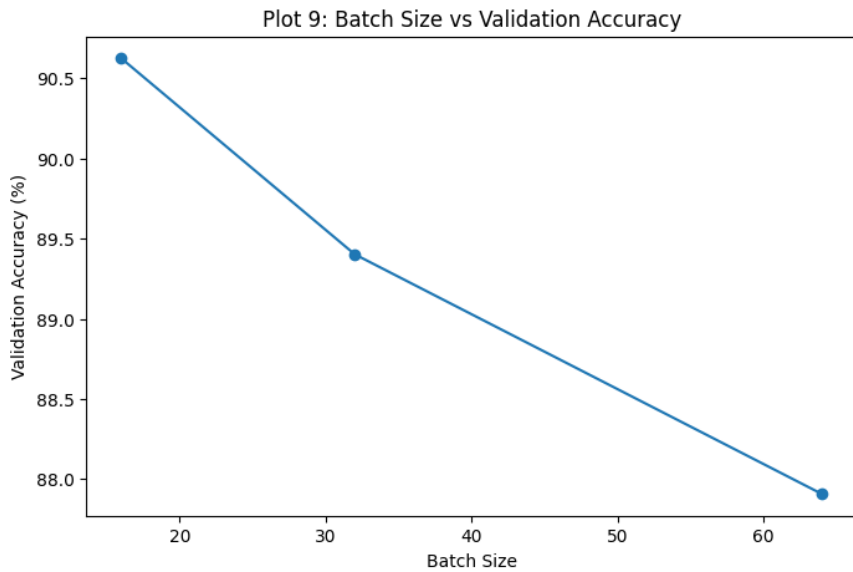


Figure 10: Batch Size vs Validation Accuracy

Batch size 16 worked best at 90.63%, followed by 32 at 89.40% and 64 at 87.91%. Smaller batches seemed to generalize

better.

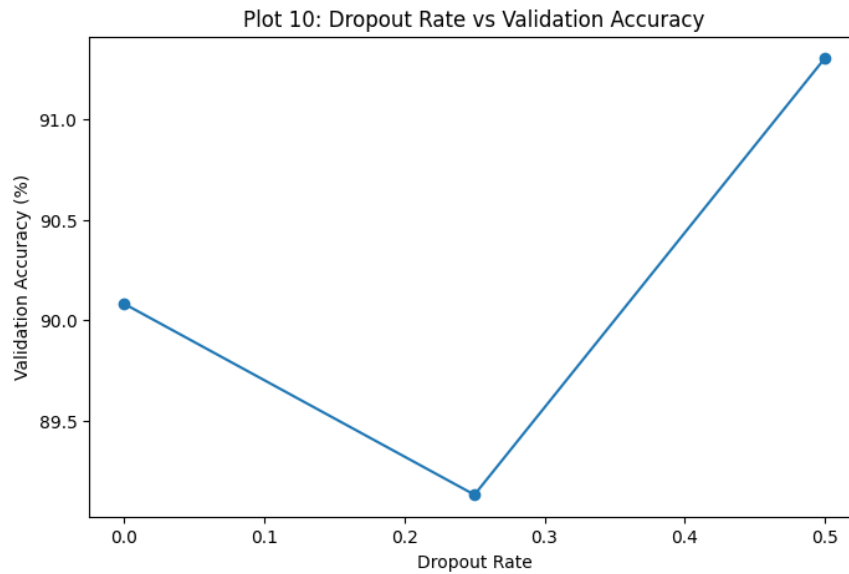


Figure 11: Dropout Rate vs Validation Accuracy

Dropout 0.5 achieved 91.30%, better than no dropout (90.08%) and 0.25 (89.13%). Higher dropout helped reduce overfitting here.

10. Transfer Learning and Fine-Tuning

MobileNetV2 pretrained on ImageNet should be used.

Case A: Feature Extraction

Pretrained MobileNetV2 → Freeze Base → New Classifier

The pretrained convolutional layers are frozen and only the newly added classification layers are trained.

Case B: Fine-Tuning

Pretrained MobileNetV2 → Unfreeze Selected Layers → Small Learning Rate

The upper layers of the pretrained network are unfrozen and trained with a smaller learning rate.

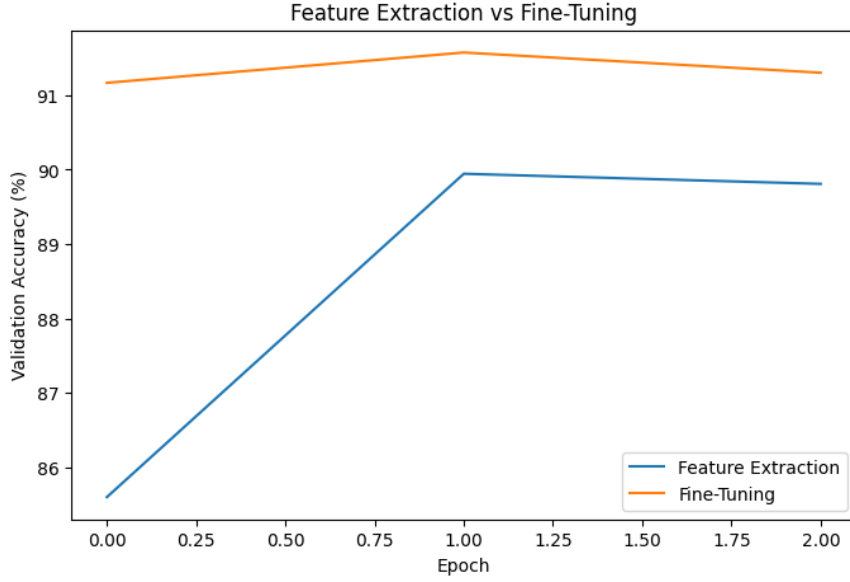


Figure 12: Feature Extraction vs Fine-Tuning (Validation Accuracy)

Fine tuning achieved a higher validation accuracy of 91.58 percent compared to 89.95 percent with feature extraction alone. This shows that unfreezing and adjusting the last layers of the pretrained model improved performance on this dataset.

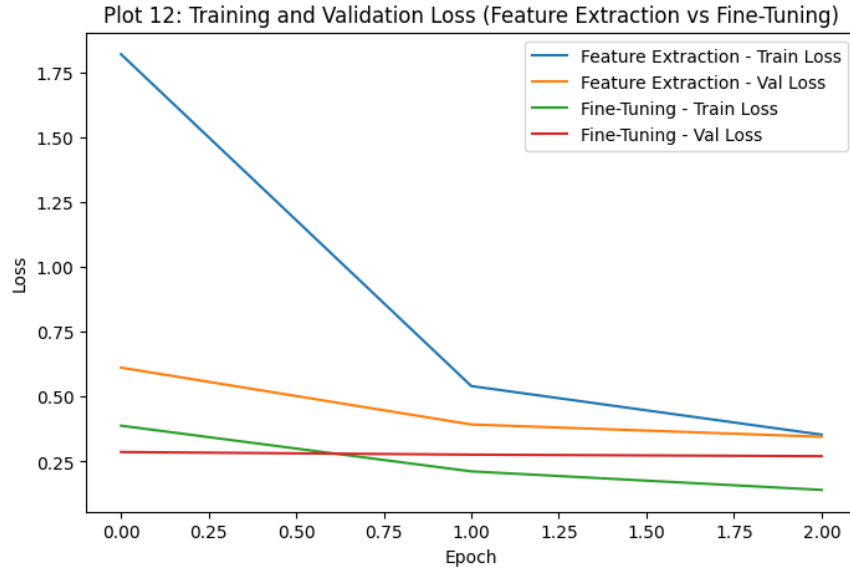


Figure 13: Training and Validation Loss (Feature Extraction vs Fine-Tuning)

Fine tuning shows a lower training and validation loss throughout compared to feature extraction alone. This confirms that allowing some of the pretrained layers to update improves the model fit for this specific dataset.

11. K-Fold Cross-Validation

After the individual studies, select promising configurations.

Use 5-fold cross-validation on the training data.

For $K = 5$,

$$\bar{A} = \frac{1}{5} \sum_{i=1}^5 A_i$$

and

$$SD = \sqrt{\frac{1}{5} \sum_{i=1}^5 (A_i - \bar{A})^2}.$$

Report the result as

$$\boxed{\bar{A} \pm SD}.$$

The independent test set must not be used during hyperparameter selection.

Configuration	F1	F2	F3	F4	F5	Mean $\pm$ SD
Best Tuned MobileNetV2	86.59%	88.79%	89.13%	86.59%	86.73%	87.57% $\pm$ 1.15%

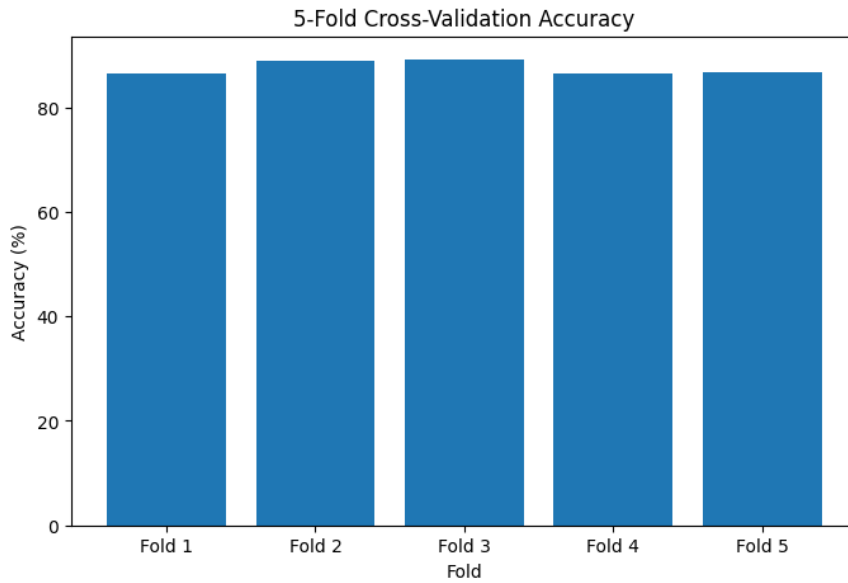


Figure 14: 5-Fold Cross-Validation Accuracy

The accuracy across the five folds ranged between approximately 86.59 and 89.13 percent, with a mean accuracy of 87.57 percent and a standard deviation of 1.15 percent. The low standard deviation indicates the model performs consistently across different data splits.

12. Final Model Evaluation

After selecting the best configuration using cross-validation:

1. Retrain the selected configuration using the complete training data.
2. Keep the independent test set untouched until this stage.
3. Evaluate the final model on the test set.

Report:

Metric	Value
Mean CV Accuracy	87.57%
CV Standard Deviation	1.15%
Test Accuracy	87.87%
Precision	88.91%
Recall	87.87%
F1-score	87.61%
Training Time	46.65 s
Number of Parameters	2,305,381

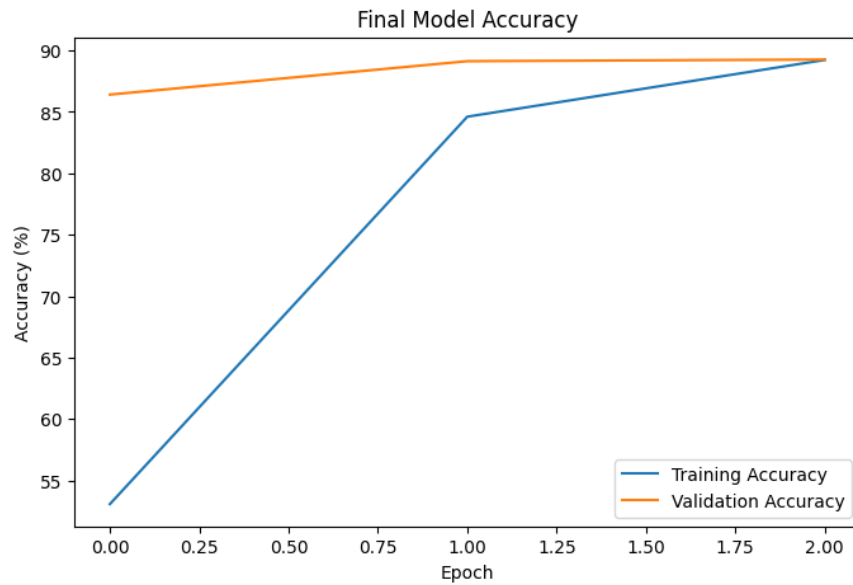


Figure 15: Final Model Accuracy

Training accuracy increased steadily across the three epochs, reaching close to 89 percent. Validation accuracy also increased and stayed close to the training accuracy, indicating the final model generalizes reasonably well without major overfitting.

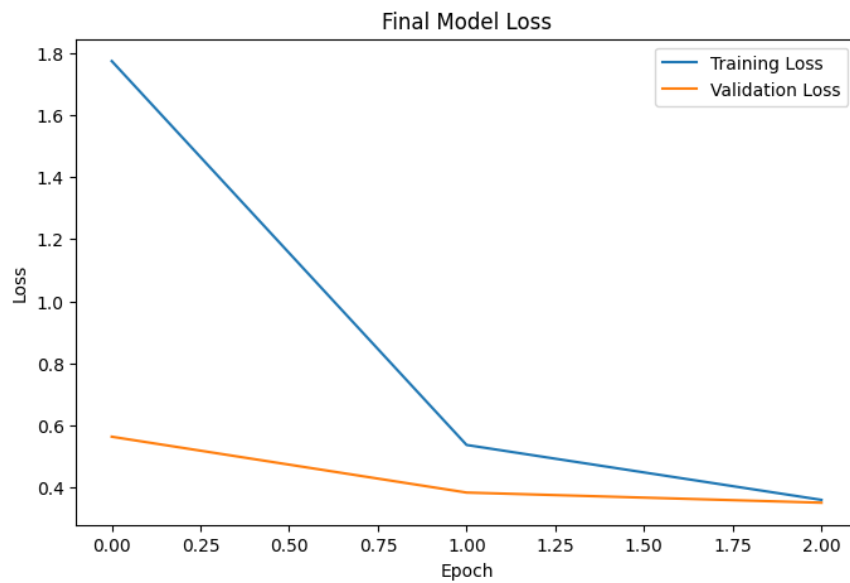


Figure 16: Final Model Loss

Training loss decreased consistently from around 1.77 to 0.36 across the three epochs. Validation loss followed a similar decreasing trend and remained close to the training loss, supporting stable convergence.

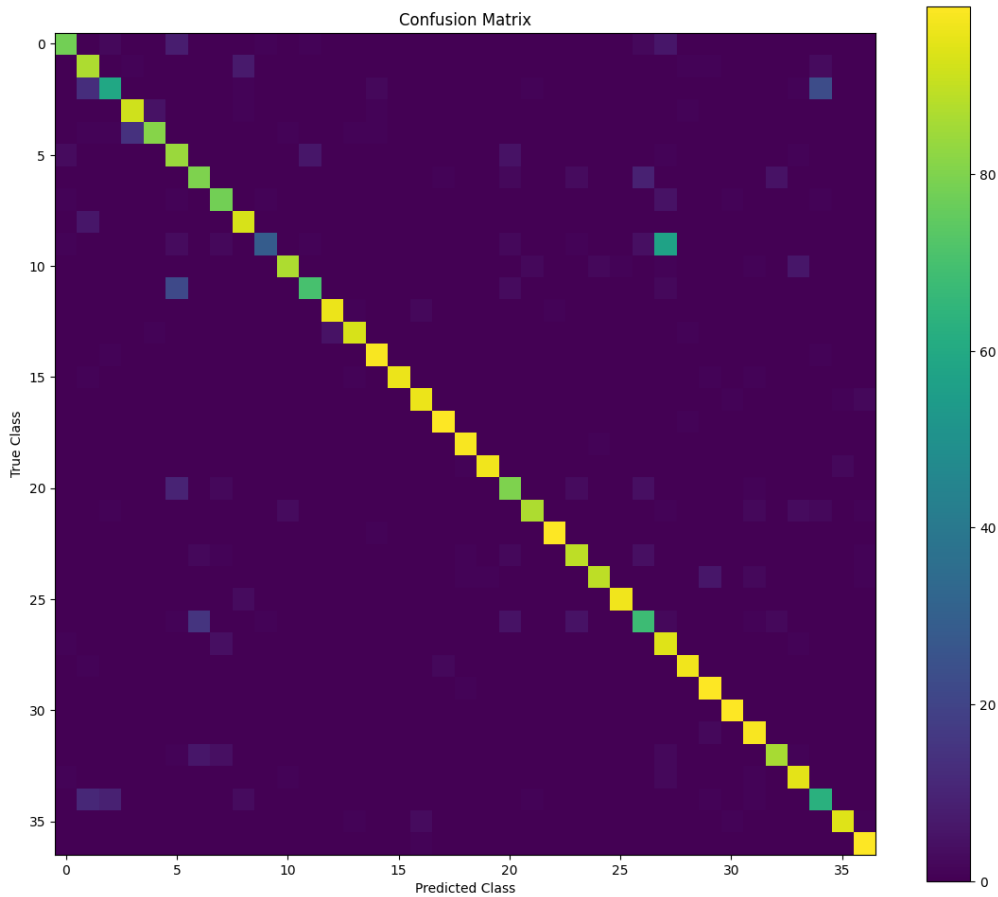


Figure 17: Confusion Matrix

Most predictions are concentrated along the diagonal, indicating correct classification for the majority of breeds. The best classified breed was scottish terrier at 100 percent accuracy, while the lowest was British Shorthair at only 29 percent, likely due to visual similarity with other cat breeds such as Russian Blue.

13. Overall Results

Configuration	CV Accuracy	SD	Test Accuracy	Training Time
Baseline	—	—	91.03%	56.53 s
Best Initialization (Zero)	—	—	90.63%	65.00 s
Best Regularization (L2)	—	—	91.98%	67.26 s
Best Optimizer (Adam)	—	—	91.44%	47.39 s
Best Hyperparameters	—	—	91.30%	61.92 s
Fine-Tuned Model	—	—	91.58%	54.41 s
Final Model (Evaluated)	87.57%	1.15%	87.87%	46.65 s

Experiment	Best Setting	Validation Accuracy (%)
Weight Initialization	Zero	90.63
Regularization	L2	91.98
Batch Normalization	Without BN	91.58
Optimizer	Adam	91.44
Learning Rate	0.001	89.40
Batch Size	16	90.63
Dropout	0.5	91.30
Feature Extraction	Frozen MobileNetV2	89.95
Fine-Tuning	0.0001	91.58

14. Discussion Questions

1. What is the difference between model parameters and hyperparameters?

Model parameters are values learned automatically by the network during training, such as weights and biases. Hyperparameters are settings chosen before training begins, such as learning rate, batch size and dropout rate.

2. Why is weight initialization important?

Good initialization helps the network start training from a reasonable point, allowing gradients to flow properly and convergence to happen faster and more reliably.

3. Why can zero initialization be problematic for neural networks?

If all weights start at zero, every neuron in a layer computes the same output and receives the same gradient during backpropagation. This means neurons never become different from each other, which limits the network's ability to learn diverse features.

4. Compare Xavier and He initialization.

Xavier initialization scales weights based on the number of input and output units and works well with sigmoid or tanh activations. He initialization uses a similar idea but is scaled specifically for ReLU based networks, since ReLU zeroes out half the activations on average.

5. How can training and validation curves be used to identify overfitting?

Overfitting is visible when training accuracy goes up but validation accuracy stays flat or falls. Similarly, if training loss drops while validation loss rises, that gap indicates overfitting.

6. How does Dropout reduce overfitting?

Dropout randomly turns off some neurons during training. This stops the network from depending too much on any single neuron, making it learn more robust features.

7. What is the purpose of Batch Normalization?

Batch Normalization normalizes mini-batch activations to have zero mean and unit variance. This helps training become more stable and faster.

8. Explain the numerical Batch Normalization example.

For $[2, 4, 6, 8]$, mean is 5 and variance is 5. After normalizing, we get $[-1.342, -0.447, 0.447, 1.342]$.

9. What are the roles of gamma and beta?

Gamma and beta are learnable parameters that scale and shift the normalized activations. They allow the network to recover the original representation if normalization alone is not optimal for learning.

10. Compare SGD, Momentum, RMSProp and Adam.

SGD updates weights using a fixed step in the gradient direction and can converge slowly. Momentum adds a moving average of past gradients to speed up convergence. RMSProp adapts the learning rate for each parameter based on recent gradient magnitudes. Adam combines momentum and RMSProp, making it generally the fastest and most stable of the four.

11. What happens when the learning rate is too large?

The model may overshoot the minimum of the loss function, causing unstable training, oscillating loss values, or in severe cases the loss diverging entirely.

12. What happens when the learning rate is too small?

Training becomes very slow, since weights update by very small amounts each step. The model may not converge to a good solution within a limited number of epochs.

13. What is the effect of increasing batch size?

Larger batch sizes provide more stable gradient estimates and can train faster per epoch due to better hardware utilization, but they may generalize slightly worse and require more memory.

14. Explain stride and padding.

Stride determines how many pixels the filter moves at each step, controlling how much the output shrinks. Padding adds extra pixels around the input border, allowing control over the output size and ensuring border information is not lost.

15. Why is MobileNetV2 computationally efficient?

MobileNetV2 uses depthwise separable convolutions and inverted residual blocks with linear bottlenecks, which significantly reduce the number of parameters and computations compared to standard convolutional networks.

16. What is depthwise separable convolution?

It splits a standard convolution into two steps, a depthwise convolution that applies a single filter per input channel, followed by a pointwise 1×1 convolution that combines the outputs. This drastically reduces computation compared to a standard convolution.

17. What is transfer learning?

Transfer learning reuses a model already trained on a large dataset, such as ImageNet, and adapts it to a new task by reusing its learned features instead of training a new network from scratch.

18. Differentiate feature extraction and fine-tuning.

Feature extraction keeps the pretrained base completely frozen and only trains the new classifier layers. Fine-tuning unfreezes some of the pretrained layers and trains them along with the classifier, usually with a smaller

learning rate.

19. **Why is a smaller learning rate generally used during fine-tuning?**

The pretrained layers already contain useful learned features. A smaller learning rate ensures these features are only slightly adjusted rather than being overwritten or destroyed during training on the new dataset.

20. **Why is K-Fold Cross-Validation useful for hyperparameter selection?**

It evaluates a configuration across multiple different train-validation splits, giving a more reliable estimate of performance and reducing the chance of selecting a configuration that only performs well due to a lucky data split.

21. **Why must the test set remain untouched during tuning?**

Using the test set during tuning would let information from it influence model selection, causing overly optimistic and unreliable estimates of how the model performs on truly unseen data.

22. **Why should mean and standard deviation both be reported?**

The mean shows the average performance of a configuration, while the standard deviation shows how consistent that performance is across different data splits. A configuration with high mean but high variability may be less reliable than one with slightly lower mean but more consistency.

23. **Is the highest validation accuracy always sufficient to select a model?**

No, a model with the highest single validation accuracy may still have high variability across folds or higher computational cost. Standard deviation and training time should also be considered before making a final selection.

15. Additional Exercise

Select two new combinations of learning rate, dropout, batch size and fine-tuning strategy. Evaluate them using 5-fold cross-validation and compare their results with the selected configuration.

Students must justify whether the new configuration is preferable based on accuracy, standard deviation, computational cost and final test performance.

16. Expected Outcome

Students should experimentally demonstrate that CNN performance is strongly influenced by initialization, regularization, optimization and hyperparameter choices. They should understand the MobileNetV2 architecture, perform transfer learning and fine-tuning, and select a reliable final configuration using 5-fold cross-validation.

17. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sergey Ioffe and Christian Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," ICML, 2015.
3. Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov and Liang-Chieh Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," CVPR, 2018.
4. Omkar M. Parkhi, Andrea Vedaldi, Andrew Zisserman and C. V. Jawahar, "Cats and Dogs," IEEE Conference on Computer Vision and Pattern Recognition, 2012.
5. TensorFlow Documentation: <https://www.tensorflow.org>
6. Keras Documentation: <https://keras.io>

18. Sources

- **GitHub Repository:** https://github.com/Saraswathi-Nalamothu/deep-learning_experiments
- **Colab Notebook:** <https://colab.research.google.com/drive/1os29Ps8uMdGD7RYnAfkcrYsXnq4npyBE?usp=sharing>